



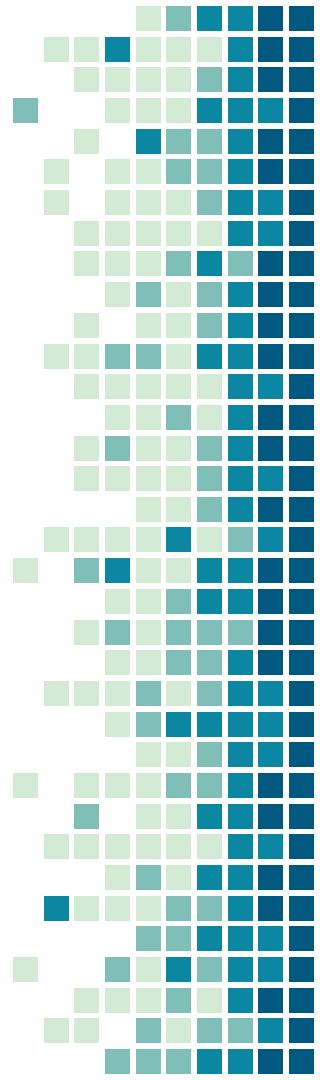
CdL in Informatica – A.A. 2024 – 2025

Programmazione 1 – Modulo 2

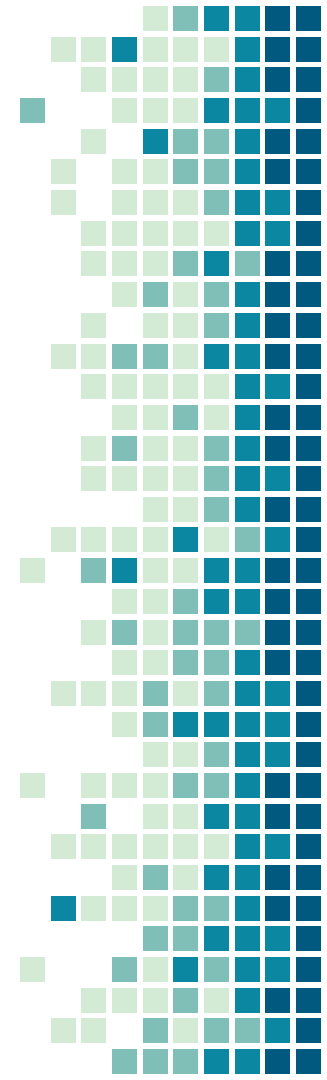
Lezione 10
14/11/2024

Andrea Loddo

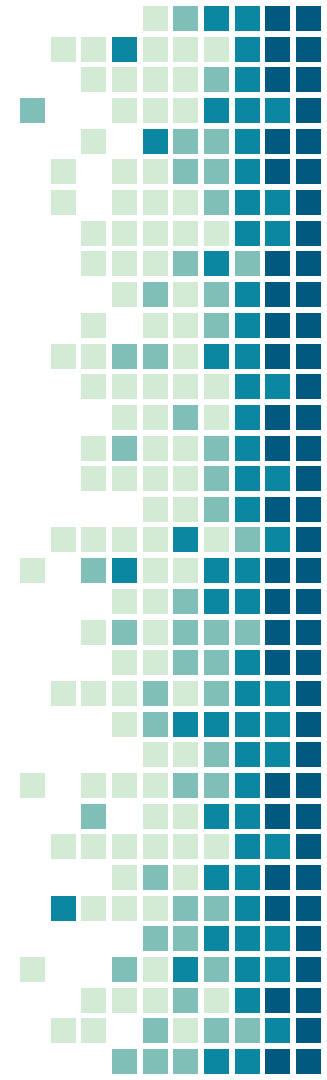
Federico Meloni - Alessandra Perniciano - Fabio Pili



Autovalutazione (LAB T)

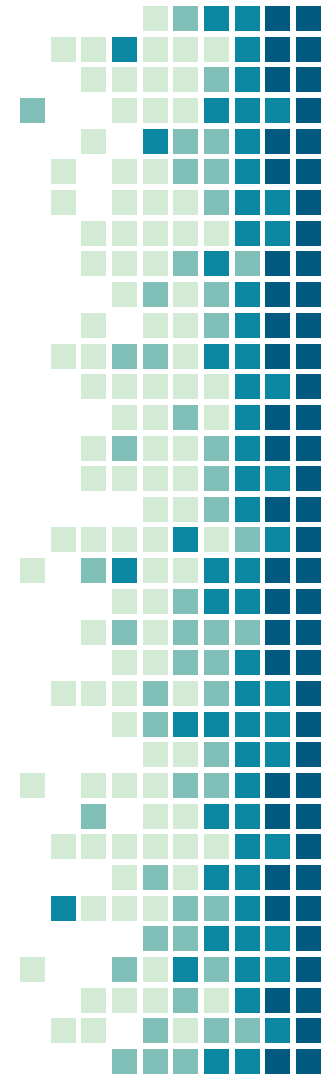


Autovalutazione (Aula Magna exFisica)



Argomenti

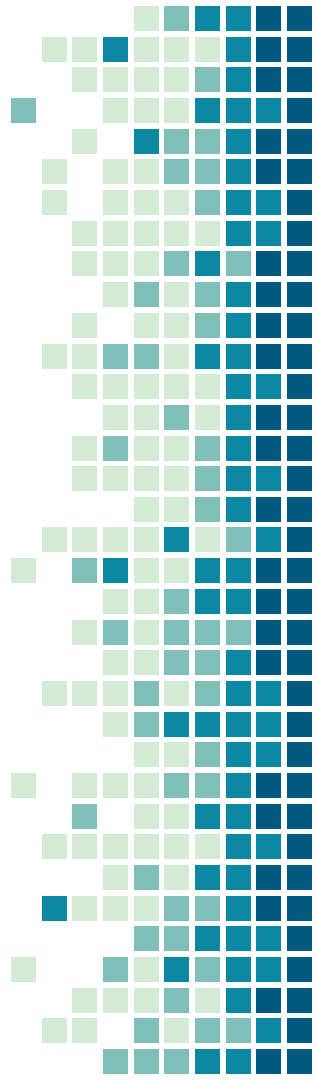
- Limiti di interi e float
- Enumerazioni
- Typedef
- Strutture
 - Dichiarazione
 - Inizializzazione
 - Accesso
 - Modifica
- Strutture annidate



Limiti del compilatore

```
#include <limits.h>
```

- La libreria limits definisce alcune costanti relative al range dei tipi int e char
- I range dipendono dal compilatore, tra parentesi i minimi richiesti dallo standard C99:
 - CHAR_BIT (numero di bit per char, in genere 8)
 - CHAR_MAX (in genere +127)
 - CHAR_MIN (in genere -127)
 - INT_MAX (almeno +32767)
 - INT_MIN (almeno -32767)
- Superare il minimo: underflow
- Superare il massimo: overflow



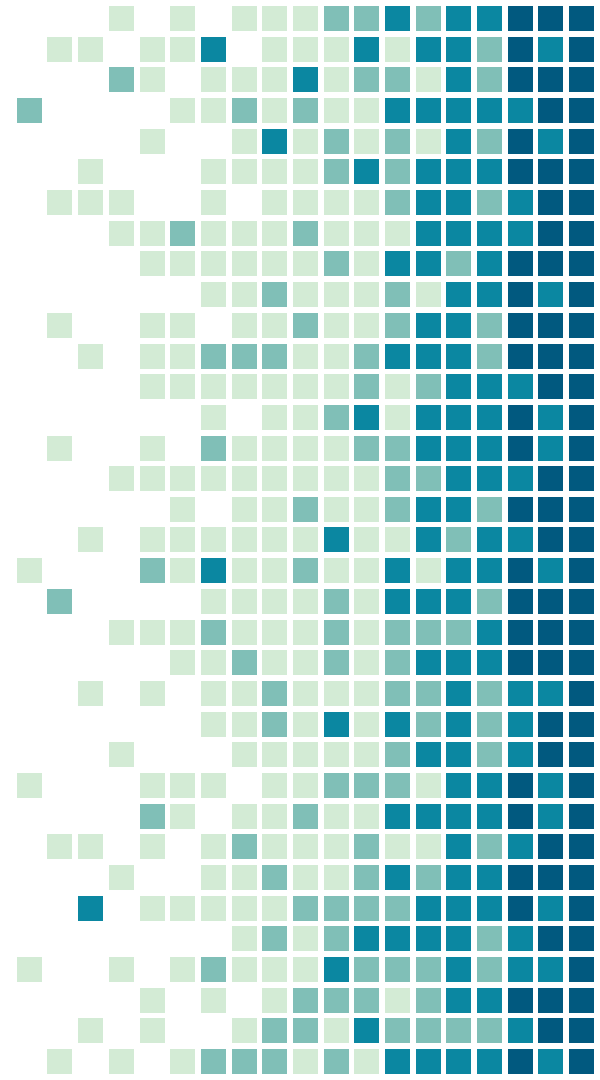
Tipi float di base e limiti

```
#include <float.h>
```

- **float**: valore in singola precisione (32 bit)
- **double**: valore in doppia precisione (64 bit)
- Lo standard C99 richiede che il tipo double debba avere almeno lo stesso numero di bit del tipo float.
- La libreria **float** definisce alcune costanti relative alle dimensioni dei tipi di dati floating-point (dipendono dal compilatore, tra parentesi i minimi richiesti dallo standard):
 - FLT_MAX (massimo numero positivo di tipo float, almeno 10^{+37})
 - FLT_MIN (minimo positivo, almeno 10^{-37})
 - DBL_MAX (almeno 10^{+37})
 - DBL_MIN (almeno 10^{-37})

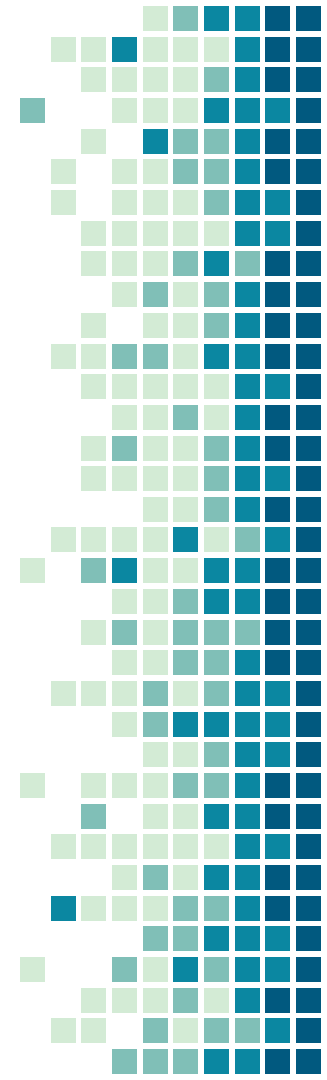
Dichiarare nuovi tipi

Enumerazioni e strutture



Definizione nuovi tipi

- Tipi di dato visti finora:
 - tipi primitivi: **int**, **float**, **double**, **char**
 - tipi non primitivi: array e stringhe
- Il linguaggio C permette di definire nuovi tipi di dato:
 - **enumerazione**: nuovo tipo definito ex-novo;
 - **struttura**: nuovo tipo contenente molteplici variabili, anche di tipi diversi tra loro.
- **Suggerimento**: quando si definiscono nuovi tipi, conviene farlo dopo le direttive di preprocessing e prima del `main( )`.



Enumerazioni

- Enumerazione: tipo di variabile che può assumere un numero **limitato** di valori **predefiniti**:
- Per definirla si usa la keyword **enum** specificando:
 - il suo **nome**
 - i **valori** possibili tra parentesi graffe.

```
enum seme {CUORI, QUADRI, FIORI, PICCHE};
```

- Crea il tipo di dato **enum seme**, che può assumere (**SEMANTICAMENTE**) come valore uno tra cuori, quadri, fiori e picche.
- Anche i valori di enumerazione sono codificati con valori **interi**:
 - CUORI vale 0, QUADRI vale 1, e così via, con incremento di passo unitario
 - È possibile alterare i valori numerici con assegnamenti:

```
enum seme {CUORI=1, QUADRI, FIORI, PICCHE};
```

Typedef

- Nel linguaggio C, la vera comodità è il dichiarare un nuovo tipo tramite la keyword **typedef**:

```
/* Forma compatta */  
typedef enum{CUORI, QUADRI, FIORI, PICCHE} Seme;  
  
/* Forma estesa */  
enum seme{CUORI, QUADRI, FIORI, PICCHE};  
typedef enum seme Seme;
```

- Esempi di enumerazioni:

```
typedef enum {LUN, MAR, MER, GIO, VEN, SAB, DOM} Giorno;  
typedef enum {SI, NO, FORSE} Scelta;  
typedef enum {MACOS, WINDOWS, LINUX} Os;
```

- La **typedef** può essere usata con qualsiasi tipo:

```
typedef int INTERO;  
typedef char CARATTERE;
```

Strutture

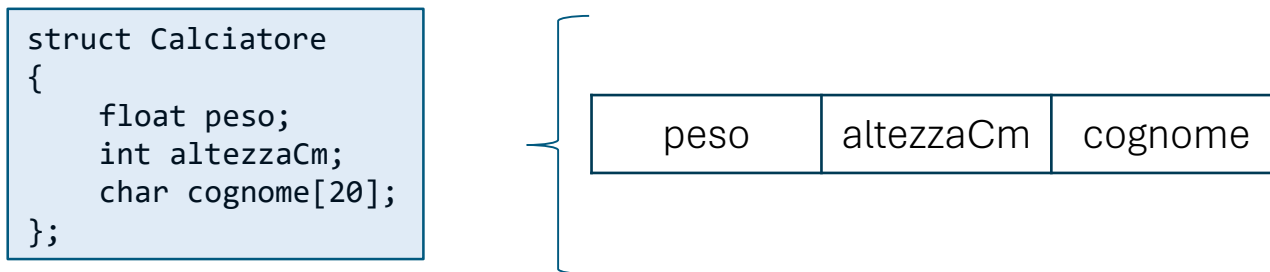
- Utili per descrivere un'entità e per salvarne lo stato durante l'esecuzione, se questo può cambiare (es: dati anagrafici).
- Una **struttura** può contenere molteplici variabili di tipi differenti.

```
/* Sintassi di definizione struttura */  
struct nomeStruttura  
{  
    tipoCampo1 nomeCampo1;  
    tipoCampo2 nomeCampo2;  
    ...  
}
```

- Sintassi: keyword **struct** seguita dal nome.
- Indicazioni:
 - i nomi dei campi devono essere univoci, come le variabili classiche
 - i tipi dei campi possono essere: primitivi, enumerazioni, definiti dal programmatore, strutture stesse, array
 - il numero di campi è arbitrario

Strutture

- **Suggerimento:** immaginare concettualmente la struttura come l'intestazione di una tabella.



- Una variabile di tipo struct Calciatore conterrà tre campi: peso, altezzaCm, cognome.
- Si può dichiarare una variabile struct Calciatore in questo modo:

```
struct Calciatore c;
```

Strutture e typedef

- Anche con le strutture, è conveniente usare il **typedef** per associare un nome al nuovo tipo (per esempio Struct Calciatore):

```
struct Calciatore    // forma estesa
{
    float peso;
    int altezzaCm;
    char cognome[20];
};
typedef struct Calciatore Calciatore;
```

```
/* forma compatta */
typedef struct
{
    float peso;
    int altezzaCm;
    char cognome[20];
} Calciatore;
```

- In entrambi in casi, sarà possibile dichiarare una variabile di questo tipo senza precisare le keyword **struct**:

```
Calciatore c;
```

Strutture: inizializzazione e modifica

- **Inizializzazione.** Si specificano i valori di tutti i campi, in ordine:

```
Calciatore c = {84.1, 187, "Riva"};
```

- **Modifiche** alla variabile struttura, mediante l'operatore **punto**, che permettere di accedere i campi sia in lettura che scrittura:

1. Assegnamenti sui singoli campi, trattati come comuni variabili:

```
c.peso = 84.1;  
scanf("%d", &c.altezzaCm);
```

2. Assegnamento tra due variabili struttura dello stesso tipo (copia):

```
Calciatore c1, c2;  
c2 = c1;
```

N.B.: cosa succede, con l'assegnamento, se la struttura contiene array (o stringhe) al suo interno?

Strutture annidate

Una struttura B può contenere una struttura A come suo campo, a patto che A sia stata dichiarata prima.

```
typedef struct
{
    float peso;
    int altezzaCm;
    char cognome[20];
}Persona;

typedef struct
{
    Persona datiCalciatore;
    int numeroMaglia;
    char ruolo;
}Calciatore;

Calciatore c;
c.numeroMaglia = 10;
c.datiCalciatore.peso = 77.9;
```

Array e strutture

- Abbiamo visto che un array può essere un campo della struttura
- Si può anche dichiarare un **array di strutture**

```
/* Esempio di dichiarazione di un array di strutture */  
Calciatore squadra[DIM];  
  
/* Esempio di accesso ai campi */  
...  
squadra[i].peso;  
squadra[i].altezzaCm;  
squadra[i].cognome;  
...
```

Subroutine e strutture

```
struct Calciatore
{
    float peso;
    int altezzaCm;
    char cognome[20];
};
typedef struct Calciatore Calciatore;
```

È possibile:

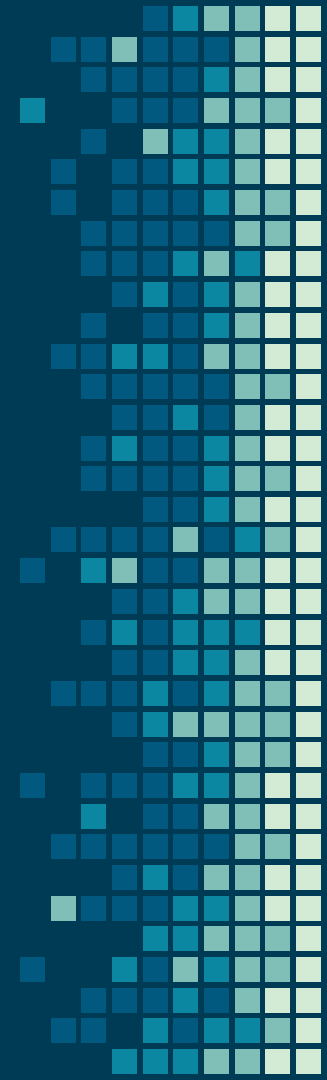
- passare una struttura come parametro a un sottoprogramma;
- realizzare un sottoprogramma che restituisca una struttura.

```
/* Esempio di passaggio di
   struttura come parametro */
int verificaAltezza(Calciatore c)
{
    if(c.altezzaCm > 200)
        return 1;
    else
        return 0;
}
```

```
/* Esempio di subroutine che
   restituisce una struttura */
Calciatore acquisisciDati( )
{
    Calciatore c;
    scanf("%f", &c.peso);
    ...
    return c;
}
```

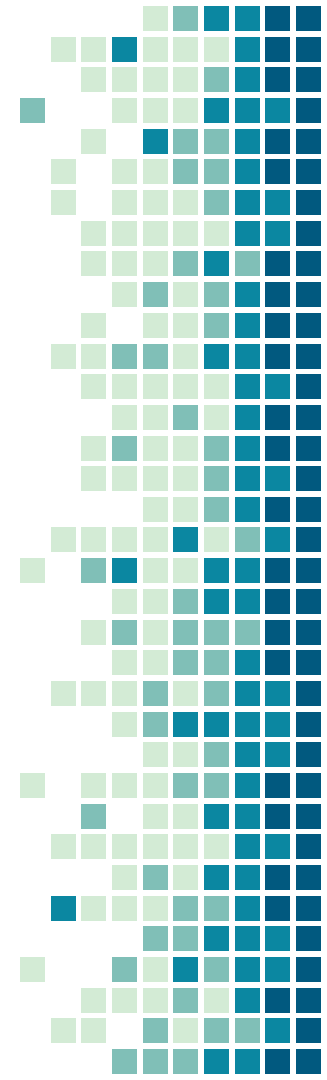
FINE!

Domande?



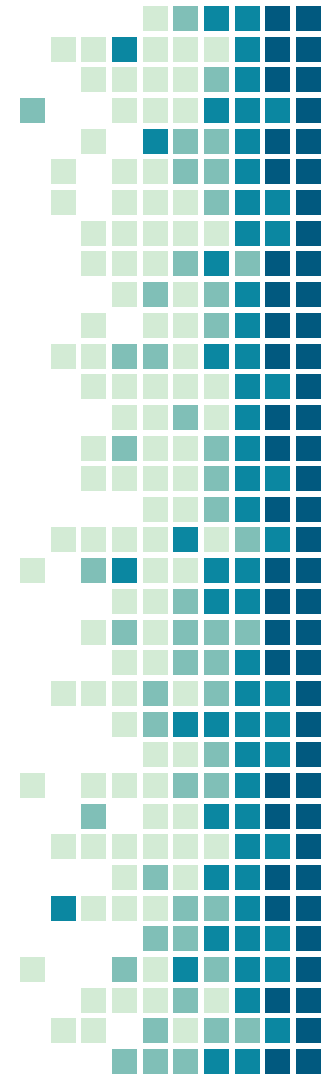
Autovalutazione

1. È possibile definire nuovi tipi in C?
2. Che differenza c'è tra enumerazione e struttura?
3. A cosa serve la keyword typedef?
4. Sono in grado di definire una nuova enumerazione?
5. La dichiarazione di una nuova enumerazione equivale a creare una nuova variabile?
6. È possibile creare due enumerazioni con lo stesso nome?
7. È possibile assegnare un valore a una enumerazione mediante scanf()? Se sì, con quale tipo?
8. Si possono avere due valori uguali nella stessa enumerazione?
9. Sono in grado di definire una nuova struttura?
10. Quali tipi di dato è possibile utilizzare all'interno dei campi di una struttura?
11. È possibile utilizzare come campo di una struttura una struttura definita precedentemente? E successivamente?
12. Quanti campi può avere una struttura?
13. La dichiarazione di una nuova struttura equivale a creare una nuova variabile?
14. Come si inizializza una variabile di tipo struttura?
15. Come si assegna un valore al campo di una struttura in una fase successiva alla dichiarazione?
16. Sono in grado di acquisire e stampare tutti i campi di una struttura?
17. In caso di acquisizione di un intero presente all'interno della struttura è necessario usare l'operatore '&'?
18. È possibile stampare tutti i campi di una struttura con una singola printf()? Se sì, come?
19. Come viene memorizzata in memoria una struttura?
20. È possibile passare una struttura a una subroutine come parametro? E riceverla come valore di ritorno?



Esercizi

1. Scrivere, in linguaggio C, la definizione di tipi di dato appropriati per:
 - i possibili gruppi sanguigni, distinguendo tra gruppo vero e proprio e Rh;
 - la sequenza dei vostri esami all'università, tenendo conto che ogni esame ha un nome, un docente, un voto e una data di svolgimento;
 - l'insieme delle taglie di un negozio di vestiti, dalla small alla tripla extra-large.
2. Scrivere un programma che definisca una struttura studente comprensiva delle seguenti informazioni: nome, cognome, matricola, corso di laurea, anno di immatricolazione e tipo di corso (triennale o magistrale).
Il programma deve svolgere le seguenti operazioni:
 - Acquisire le informazioni relative a cinque studenti e memorizzarle in un array.
 - Richiedere all'utente di inserire il numero di matricola di uno studente di cui cercare le informazioni nell'array.
 - Cercare lo studente e stampare in output tutti i suoi dati se la ricerca ha prodotto risultati, oppure indicare che non è stato trovato uno studente con tale matricola.



Esercizi

3. Scrivere un programma in cui viene definita una struttura NumeroComplesso, atta a contenere due variabili reali:
- parte reale;
 - parte immaginaria.
 - Il programma deve dichiarare una variabile di tipo NumeroComplesso, acquisire in input dall'utente le due parti del numero, effettuare il calcolo del modulo del numero e stampare le due parti del numero e il modulo calcolato. NB: dato un numero complesso z , la sua parte reale x e immaginaria y , il modulo si calcola come: $|z| = \sqrt{x^2 + y^2}$
4. Definire una enumerazione che rappresenti il gruppo sanguigno GS (A, B, ZERO, AB). Definire poi una struttura che contenga i dati di una Persona (età, peso in kg, altezza in cm) e successivamente una struttura Donatore che contenga un campo Persona, un campo enumerazione GS, un intero, rappresentante il numero di giorni trascorsi dall'ultima donazione, e un intero rappresentante il codice identificativo del donatore di sangue. Realizzare un programma in cui viene dichiarato un array di donatori di lunghezza a piacere e, dopo aver inserito i dati di ogni donatore, il programma deve stabilire se ognuno di essi può donare il sangue. In particolare:
- Se il peso è inferiore a 50 kg, non è possibile
 - Se l'ultima donazione è stata fatta prima di 90 giorni, non è possibile.
 - Se l'età è inferiore a 18 anni o maggiore di 65, non è possibile.
 - È possibile in tutti gli altri casi.
 - Stampare, come output, tutti i dati dei pazienti che possono donare il sangue, precisando il perché è possibile.

Esercizi

5. Risolvere il seguente esercizio, composto dai seguenti sotto esercizi:

1. Scrivere un'enumerazione `GiornoSettimana` che rappresenti i 7 giorni della settimana (da LUN a DOM) e un'enumerazione `Mese` che rappresenti i 12 mesi dell'anno (da GEN a DIC)
2. Scrivere un tipo di dato struttura `Data` composto dai seguenti quattro campi: `giorno`, `giornoSettimana`, `mese` e `anno`. `Giorno` è di tipo intero, `giornoSettimana` è di tipo enumerazione `GiornoSettimana`, `mese` è di tipo enumerazione `Mese`, mentre `anno` è un intero.
3. Scrivere un tipo di dato struttura `Persona`, costituito dai campi `nome`, `cognome`, `luogo di nascita`, `data di nascita`. I primi tre campi sono stringhe, mentre l'ultimo è di tipo `Data`.
4. Scrivere una funzione **`acquisisciPersona`**, che dichiari una struttura `Persona` e richieda in input all'utente i dati d'interesse. Effettuare i controlli ritenuti opportuni per acquisire date valide. Tale funzione deve restituire la struttura appena acquisita.
5. Scrivere una procedura **`stampaDatiPersona`** che, ricevuta una `Persona` come parametro, stampi tutti i suoi dati d'interesse.
6. Nel main, dichiarare una struttura `Persona`, a cui assegnare l'output di **`acquisisciPersona`**; dopodiché stampare i suoi dati di interesse mediante la procedura **`stampaDatiPersona`**.